



同濟大學
TONGJI UNIVERSITY

数据仓库课程项目报告

小组成员: 2351299 王 雷

小组成员: 2351129 黄景胤

小组成员: 2354185 周 达

目录

1.数据存储及优化对比	3
1.1 整体存储模式设计说明	3
1.1.1 各存储系统的职责划分	3
openGauss（关系型数据仓库）	3
Hive + HDFS（分布式文件系统）	4
Neo4j（图数据存储）	5
2.2 关系型数据库	5
2.2.1 适用查询范围	5
2.2.2 存储优化工作	6
2.2.3 优化前后对比	8
2.3 分布式存储	10
2.3.1 适用查询范围	10
2.3.2 存储优化工作	11
2.3.3 优化前后对比	12
2.4 图数据库	13
2.4.1 适用查询范围	13
2.4.2 存储优化工作	14
2.4.3 优化前后对比	14
3.数据质量保证	16
3.1 数据质量保障方法	16
3.2 数据质量影响因素	16
5.日志记录与审计	17
6.小组分工与贡献度	18

1.数据存储及优化对比

1.1 整体存储模式设计说明

本项目采用多存储系统协同的数据仓库架构，使用关系型数据库、分布式数据库以及图数据库进行查询，实现对不同类型的数据存储方式查询性能的对比以及同种数据存储方式查询性能对比。前端采用 React 实现数据可视化与交互，后端通过 Flask 统一调度多种存储系统，对外提供一致的数据访问接口。

1.1.1 各存储系统的职责划分

openGauss（关系型数据仓库）

定位：

openGauss 在本项目中承担核心结构化数据仓库的角色，主要服务于交互式查询与中小规模统计分析。

存储内容：

- 电影维度信息 (dim_movies)
- 演员维度信息 (dim_actors)
- 电影 – 演员关联关系 (movie_actor)
- 用户评分与评论事实数据 (fact_reviews)

设计动机：

这些数据结构稳定、字段明确

查询频率高，通常用于前端直接展示

数据量相对可控，适合关系型数据库处理

典型适用查询：

按电影名称、导演、类型查询电影信息

按评分区间筛选电影

统计某部电影的平均评分、评论数量

支持分页、排序等交互式操作

Hive + HDFS（分布式文件系统）

定位：

Hive + HDFS 用于存储大规模、明细级、以分析为主的数据，承担离线分析与批处理计算任务。

存储内容：

- 清洗后的评论明细数据 (reviews_clean_amazon)
- 电影完整元数据 (movies_meta_dw)
- 数据来源与溯源字段 (source_files)

设计动机：

评论数据规模大，且随时间持续增长

分析任务多为全表扫描或时间范围扫描

数据主要“读多写少”，适合列式存储

典型适用查询：

按年 / 月统计评论数量

长时间跨度的评分趋势分析

跨来源（Amazon / Snap 等）数据统计

批量文本分析或特征提取

Neo4j (图数据存储)

定位:

Neo4j 用于建模并分析复杂实体关系网络, 弥补关系型数据库在多跳关联查询方面的不足。

存储内容:

- Movie / Actor / Director / User 节点
- 演员参演关系
- 导演 – 演员合作关系
- 用户 – 电影评分关系

设计动机:

演员与演员之间的合作关系天然构成图结构

多跳查询在关系型数据库中成本较高

图模型更直观地表达合作网络和路径关系

典型适用查询:

经常合作的演员有哪些

某导演常合作的演员组合

演员之间的合作路径分析

2.2 关系型数据库

2.2.1 适用查询范围

关系型数据库在处理高度结构化数据、支持复杂查询以及保证数据一致性与事务性方面具有明显优势, 适用于需要精确统计、频繁查询以及强一致性要求的业务

场景。

关系型数据库基于表结构进行数据建模，数据以行和列的形式存储在多个表中，每个字段具有明确的数据类型和约束条件。通过结构化查询语言（SQL），可以高效完成筛选、聚合、分组、排序以及多表关联等操作，因此非常适合用于存储和分析结构稳定、语义清晰的数据。

在本项目中，关系型数据库主要用于存储：

- 电影的基础结构化信息（电影名称、上映时间、评分等）
- 演员、导演等实体的基本属性
- 电影与演员、导演之间的关联关系
- 评论的评分、时间等可用于统计分析的字段

由于前端页面中存在大量基于条件筛选和统计的查询需求，例如：

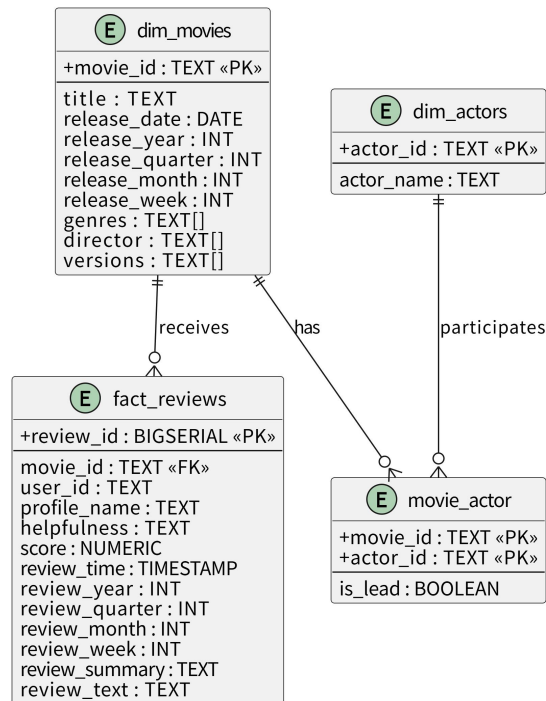
- 按年份、类型统计电影数量
- 查询某导演或演员参与的电影列表
- 统计电影平均评分、评论数量等

这些查询对响应时间要求较高，因此关系型数据库成为前端高频访问数据的主要支撑。

2.2.2 存储优化工作

在本项目中，我们针对关系型数据库的物理存储模型进行了多方面的优化，以提升查询效率并降低系统整体开销。

1. 事实表与维度表分离



项目在逻辑建模阶段采用了**事实表与维度表分离**进行设计：

- 以电影作为核心事实对象
- 将演员抽象为维度表
- 通过事实表与维度表之间的关联关系进行分析

这种建模方式在一定程度上引入了数据冗余，但显著降低了查询时的 Join 复杂度，更适合数据仓库中的统计分析场景。

2. 字段类型与存储设计优化

1. 时间字段设计

在评论表中，评论时间并未采用 `datetime` 类型直接存储，而是统一使用 `bigint` 类型的时间戳表示。这样做的原因包括：

- 保证跨系统、跨语言处理时间数据的一致性
- 相较 `datetime` 类型，占用存储空间更小
- 在范围比较和排序时，整数比较性能更优

2. 评分字段设计

在电影表中，电影评分等字段使用 float 类型进行存储。由于电影评分本身不需要极高的精度，使用 float 相较 double 可以有效减少存储空间消耗，在大规模数据场景下具有一定优势。

3. 时间维度表设计

在时间维度表中，年份、月份、星期等字段采用 varchar 类型进行存储。这种设计允许直接存储非标准或不完整的时间信息，同时避免了不同系统之间时间格式解析不一致的问题，并且在这些字段上建立索引以加速查询。

3. 冗余存储策略

1. 在电影表中冗余存储电影对应的评论数量，避免在统计热门电影或演员时频繁与评论表进行 Join 操作。
2. 将时间信息单独抽象为时间维度表，将时间戳拆分为多个粒度字段（年、月、季度等），使得查询时可以直接基于字段匹配而无需额外的时间转换计算。
3. 对评论情感分析结果进行了两次优化，最终选择在电影表中直接存储正面情感评论比例，避免在查询时对百万级评论数据进行实时聚合计算。

4. 索引与视图设计

1. 对演员姓名、导演姓名、电影类型等高频查询字段建立单列索引
2. 在时间表中建立年份索引、年 - 月组合索引、年 - 季度组合索引
3. 建立演员 - 演员合作视图、演员 - 导演合作视图，用于支持复杂关系查询

2.2.3 优化前后对比

在未进行存储优化之前，关系型数据库中的部分查询需要频繁对评论表与电影表进行关联操作，尤其是在涉及评分统计或评论数量计算时，由于 fact_reviews 表

数据量较大，查询性能受到明显影响。

例：在查询某电影的评论列表时，优化前需要对评论列表以评论时间进行降序排序，排序耗时较长。

```
REVIEWS_BY_MOVIE_ID = ""
... SELECT
...     review_id,
...     movie_id,
...     user_id,
...     profile_name,
...     helpfulness,
...     score,
...     review_time,
...     review_summary,
...     review_text
... FROM fact_reviews
... WHERE movie_id = %s
... ORDER BY review_time DESC
... LIMIT %s OFFSET %s;
... ..
```

OpenGauss	SUCCESS	40.33s
OpenGauss	SUCCESS	40.47s
OpenGauss	SUCCESS	42.62s
OpenGauss	SUCCESS	42.81s
OpenGauss	SUCCESS	38.09s
OpenGauss	SUCCESS	37.93s

在引入以下优化措施后：

- 对高频查询字段建立索引
- 合理拆分维度表与事实表
- 将演员参演关系显式存储为关联表
- 减少前端请求中不必要的多表 Join

关系型数据库在实际测试中的查询性能得到了明显提升。

例：在使用了带有评论时间降序索引之后，查询性能显著提升：

```
57 | CREATE INDEX IF NOT EXISTS idx_fact_movie_time ON fact_reviews(movie_id, review_time DESC); 41.3s
```

OpenGauss	SUCCESS	0.0474s
OpenGauss	SUCCESS	0.0321s
OpenGauss	SUCCESS	0.0224s
OpenGauss	SUCCESS	0.0241s
OpenGauss	SUCCESS	0.0258s
OpenGauss	SUCCESS	0.0271s
OpenGauss	SUCCESS	0.0235s
OpenGauss	SUCCESS	0.0227s
OpenGauss	SUCCESS	0.3850s

优化后通过索引与规范化的关联表设计，查询响应时间明显缩短，更加满足前端实时交互的性能要求。

总体来看，经过存储优化后，openGauss 能够稳定支撑 React 前端的高频查询请求，并与 Hive 和 Neo4j 形成清晰分工，为系统整体性能提供可靠保障。

2.3 分布式存储

2.3.1 适用查询范围

分布式存储系统主要面向数据规模较大、访问模式以分析型查询为主的应用场景，其核心优势在于能够通过横向扩展来支撑海量数据的存储与计算需求。在本项目中，Hive 构建在 HDFS 之上，主要用于存储评论明细数据和电影元数据等体量较大、更新频率较低的数据，为离线分析和批处理计算提供支撑。

与关系型数据库侧重交互式查询不同，Hive 更适合执行全表扫描、时间范围扫描以及大规模聚合分析任务。例如，在进行跨年份的评论数量统计、评分趋势变化分析或不同数据来源之间的对比分析时，Hive 能够充分利用分布式计算资源，在可接受的时间内完成计算任务。由于这类查询通常不直接面向前端实时交互，因此对查询延迟的要求相对较低，更关注整体吞吐能力和计算稳定性。

在本项目中，Hive 主要承担以下分析类查询任务：对评论数据按年、月等时间

粒度进行统计分析；对大规模评论数据进行聚合计算；以及对多来源数据进行统一分析。

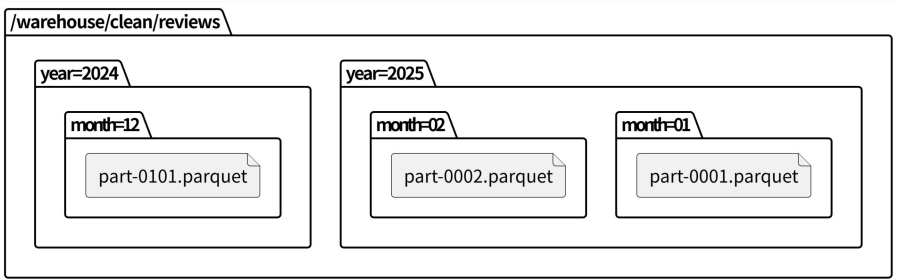
2.3.2 存储优化工作

1. 宽表结构

在 Hive 存储层面，本项目围绕数据组织方式和查询执行效率两个方面进行了优化设计。首先，在数据建模阶段，针对评论数据采用了宽表结构，将评论文本、评分、时间戳以及来源信息等字段集中存储，减少查询过程中频繁 Join 不同表所带来的性能开销。这种设计方式虽然在一定程度上增加了单表宽度，但更符合 Hive 面向分析型查询的使用特性。

2. 列式存储

其次，在表的物理存储格式上，采用了列式存储格式对数据进行组织。列式存储能够在执行聚合查询或只访问部分字段时显著减少磁盘 I/O，提高扫描效率，尤其适用于评分统计、时间趋势分析等典型分析场景。同时，在数据加载阶段对评论数据进行了统一清洗与规范化处理，保证字段类型的一致性，从而减少运行时类型转换带来的额外开销。



3. 原始时间戳保留

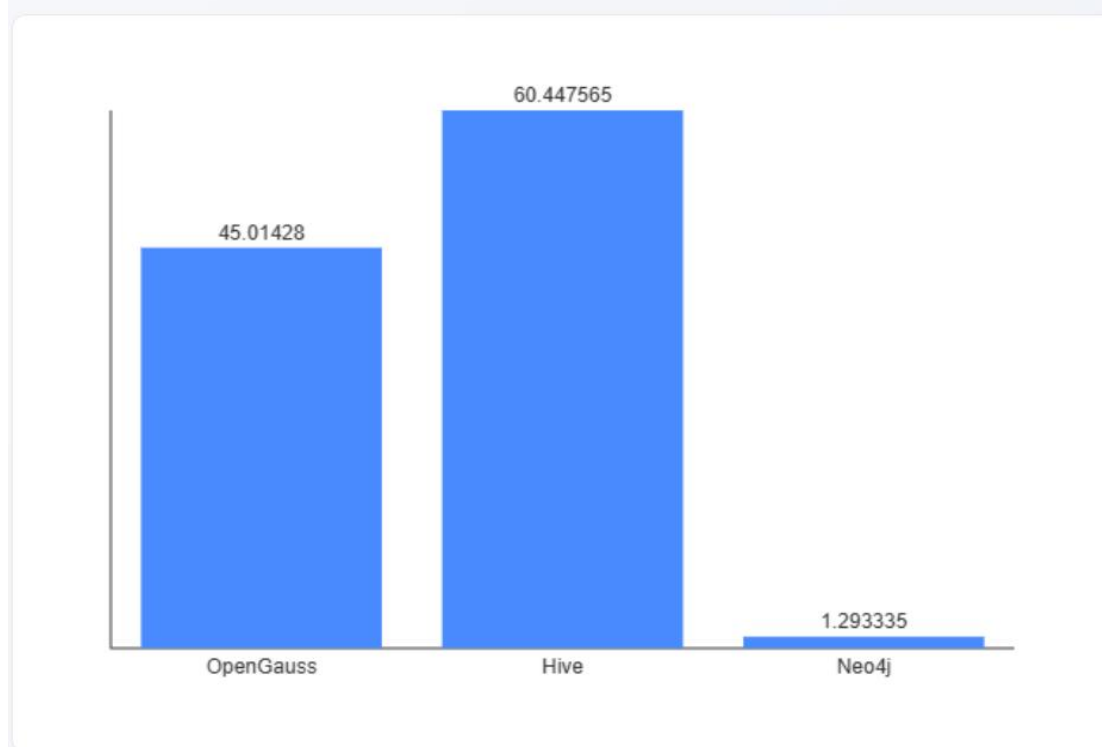
在时间相关字段的设计上, Hive 表中统一保留了原始时间戳字段, 并在 ETL 阶段派生出年、月等分析常用字段。这种做法使得后续查询可以直接基于派生字段进行过滤和分组, 而无需在查询过程中反复进行时间函数计算, 从而进一步提升查询效率。

2.3.3 优化前后对比

在引入宽表设计、列式存储格式以及时间派生字段后, Hive 的分析查询性能得到了明显改善。以按年份统计评论数量为例, 优化前查询需要进行全表扫描并执行时间函数计算, 而优化后查询可以直接基于年份字段进行分组统计, 执行效率显著提升。

例如, 在使用了宽表之后, 对于具有 6661 条电影数据的按年份查询, Hive 的查询使用了 60.45s, 鉴于其在服务器端运行启动时间需要调度 spark 资源、为了服务器内存不会爆栈限制了一部分资源的使用、测试时查空表都会出现需要 1 秒到 8 秒不等, 等情况, 除去服务器性能限制, 没有加评论索引、需要和评论表做 Join 操作的 Hive 的查询时间应该和加了时间维度字段和评论索引的 OpenGauss 的性能差不多, 足以说明通过宽表设计提高性能的作用。

各数据源执行时间对比



整体来看，通过针对分析型查询特点进行存储与结构优化，Hive 能够更加高效地支撑大规模数据分析任务，并与 openGauss 形成“实时查询 + 离线分析”的互补关系，为系统提供稳定的数据分析能力

2.4 图数据库

2.4.1 适用查询范围

图数据库主要适用于处理实体之间关系复杂、关联路径不固定的查询场景。在本项目中，电影、演员、导演和用户之间存在大量多对多关系，并且查询需求往往涉及多跳关联，例如分析演员之间的合作关系或探索某位导演的合作网络。这类查询在关系型数据库中通常需要多次 Join 操作，查询逻辑复杂且性能开销较大。Neo4j 通过图模型将实体表示为节点、关系表示为边，使得关系本身成为一等公民。基于这种数据模型，可以更加直观地表达实体之间的关联结构，并通过图遍

历算法高效完成多跳路径查询。因此，Neo4j 主要用于支撑关系探索类查询，而不承担高频统计或批量分析任务。

2.4.2 存储优化工作

在 Neo4j 的建模过程中，本项目对节点类型和关系类型进行了明确划分，确保图结构语义清晰。电影、演员、导演和用户分别建模为独立节点类型，不同实体之间通过语义明确的关系进行连接，例如“参演”“执导”“评分”等。这种设计方式不仅有助于提升查询的可读性，也为后续关系分析提供了清晰的数据基础。在属性设计方面，节点仅保留用于关系查询和路径分析所需的核心属性，避免将大量冗余或分析型字段引入图数据库，从而控制图规模并降低遍历成本。同时，在高频访问的节点属性（如演员姓名、电影名称）上建立索引，使得图查询在定位起始节点时能够快速命中目标，减少不必要的全图扫描。

2.4.3 优化前后对比

例如，再进行千数量级的电影查询时，本身节点就有时间数据的 Neo4j 在电影-演员-评论关系连接查询方面对比其他两种数据库具有非常大的优势：

```
▼ results: {,...}
  ▼ hive: {data: [{m.director: ["PGA Tour Productions"], m.genres: null, m.movie_id: "B000RYQWI8",...},...]}
    ▼ data: [{m.director: ["PGA Tour Productions"], m.genres: null, m.movie_id: "B000RYQWI8",...},...]
      ▶ [0 ... 99]
      ▶ [100 ... 199]
      ▶ [200 ... 299]
      ▶ [300 ... 399]
      ▶ [400 ... 499]
      ▶ [500 ... 599]
      ▶ [600 ... 699]
      ▶ [700 ... 799]
      ▶ [800 ... 899]
      ▶ [900 ... 999]
      ▶ [1000 ... 1099]
      ▶ [1100 ... 1199]
      ▶ [1200 ... 1299]
      ▶ [1300 ... 1312]
      error: null
      execution_time: 60.698654
      success: true
```


力。

3.数据质量保证

3.1 数据质量保障方法

数据质量是数据仓库系统稳定运行和分析结果可靠性的基础。本项目在数据采集、清洗和存储阶段均引入了相应的数据质量保障措施，以尽可能减少数据缺失、重复和不一致问题的出现。

在数据进入数据仓库之前，首先对原始数据进行完整性和合法性校验，例如检查关键字段是否为空、评分值是否落在合理区间内等。对于存在明显异常的数据，采取过滤或修正策略，确保进入仓库的数据符合基本质量要求。在数据清洗阶段，对来自不同数据源的数据进行统一格式规范，解决字段命名不一致、时间格式不同等问题，从而保证后续分析的一致性。

在存储阶段，通过在关系型数据库中设置主键、外键约束以及必要的唯一性约束，防止重复数据和非法关联关系的产生。同时，通过日志记录和定期抽样检查的方式，对数据质量进行持续监控，及时发现并修正潜在问题。

3.2 数据质量影响因素

在实际数据处理过程中，数据质量受多种因素影响。首先，不同数据来源的数据标准和质量参差不齐，可能导致字段缺失或语义不一致的问题。其次，在数据采集和传输过程中，网络异常、程序错误以及网站反爬机制也可能导致数据丢失或重复。最后，在多系统协同的架构下，数据在不同存储系统之间流转，如果缺乏统一的规范和校验机制，也可能造成数据不一致。

针对上述影响因素，本项目通过统一数据模型、规范 ETL 流程以及加强数据校

验等方式进行控制，从而在整体上提升数据仓库的数据质量水平。

4.数据血缘使用场景

在本项目中，数据血缘主要用于描述数据从原始数据源到最终分析结果之间的流转过程。通过记录数据的来源、清洗规则以及存储位置，可以清晰地追踪某一分析结果所依赖的原始数据和中间处理步骤。

比如，在 ETL 流程中，通过数据血缘可以记录删除的非电影 asin 的数量，以及对哈利波特系列电影在整个数据处理流程中的来源、加工路径和加工后去向进行追踪。

```
===== 总计：共删除 62397 个非电影 ASIN =====  
哈利波特电影数（合并后）：{'harry_potter_movies': 71}  
哈利波特版本数（网页行）：{'harry_potter_versions': 116}  
哈利波特第一部合并后网页数：{'hp1_webpages': 15}
```

数据血缘在问题定位和结果解释方面具有重要作用。当分析结果出现异常时，可以通过数据血缘快速定位问题可能产生的环节，例如数据采集、清洗或聚合过程，从而提高问题排查效率。同时，在进行数据模型调整或存储结构优化时，数据血缘信息也有助于评估改动可能带来的影响，避免对下游分析产生不可控的副作用。

5.日志记录与审计

为了保障系统运行的可追溯性与安全性，本项目在后端服务层引入了日志记录机制，对用户查询请求、参数信息以及各存储系统的执行情况进行统一记录。日志内容包括查询类型、请求时间、目标存储系统以及执行结果等信息，为系统运行

状态分析和性能评估提供数据支持。

```
publicsql • init.sql • .gitignore M • query.log M X
backend > logs > query.log
1 2025-12-27 17:22:42,707 | INFO | [QUERY] cb39ea59 | OpenGauss | SUCCESS | 0.0274s
2 2025-12-27 17:22:42,763 | INFO | [QUERY] 61b572b4 | Hive | FAILED | 0.0283s | ERROR: TExecuteStatementResp(status=TStatus(statusCode=3, i
3 2025-12-27 17:22:43,616 | INFO | [QUERY] 1c8f3bae | Neo4j | SUCCESS | 0.7300s
4 2025-12-27 17:33:41,397 | INFO | [QUERY] 1c8f3bae | Neo4j | SUCCESS | 0.0184s
5 2025-12-27 17:33:41,422 | INFO | [QUERY] cb39ea59 | OpenGauss | SUCCESS | 0.0290s
6 2025-12-27 17:33:42,222 | INFO | [QUERY] 61b572b4 | Hive | FAILED | 0.7596s | ERROR: cannot convert dictionary update sequence element #0
7 2025-12-27 17:40:27,817 | INFO | [QUERY] 1c8f3bae | Neo4j | SUCCESS | 0.0196s
8 2025-12-27 17:40:27,838 | INFO | [QUERY] cb39ea59 | OpenGauss | SUCCESS | 0.0275s
9 2025-12-27 17:40:28,629 | INFO | [QUERY] 61b572b4 | Hive | SUCCESS | 0.7349s
10 2025-12-27 18:06:59,177 | INFO | [QUERY] 586fa63d | Neo4j | SUCCESS | 0.0524s
11 2025-12-27 18:06:59,202 | INFO | [QUERY] 572c3411 | OpenGauss | SUCCESS | 0.0322s
12 2025-12-27 18:06:59,870 | INFO | [QUERY] 2c5e8eec | Hive | SUCCESS | 0.6248s
13 2025-12-27 18:07:10,617 | INFO | [QUERY] 74949aa7 | OpenGauss | SUCCESS | 0.0501s
14 2025-12-27 18:07:10,640 | INFO | [QUERY] 32172faa | Neo4j | SUCCESS | 0.0221s
15 2025-12-27 18:07:11,252 | INFO | [QUERY] d0b49b7a | Hive | SUCCESS | 0.5595s
16 2025-12-27 18:07:18,738 | INFO | [QUERY] 434b15ab | OpenGauss | SUCCESS | 0.0289s
17 2025-12-27 18:07:18,762 | INFO | [QUERY] b1cac639 | Neo4j | SUCCESS | 0.0172s
18 2025-12-27 18:07:19,466 | INFO | [QUERY] 94d3cfd9 | Hive | SUCCESS | 0.6310s
19 2025-12-27 18:07:22,427 | INFO | [QUERY] 658e721f | OpenGauss | SUCCESS | 0.0783s
20 2025-12-27 18:07:22,453 | INFO | [QUERY] 81a139fd | Neo4j | SUCCESS | 0.0247s
21 2025-12-27 18:07:23,134 | INFO | [QUERY] 96990995 | Hive | SUCCESS | 0.6362s
22 2025-12-27 18:07:26,149 | INFO | [QUERY] 36b272a7 | OpenGauss | SUCCESS | 0.0303s
23 2025-12-27 18:07:26,178 | INFO | [QUERY] 96b4d912 | Neo4j | SUCCESS | 0.0166s
24 2025-12-27 18:07:26,867 | INFO | [QUERY] 4bc759f6 | Hive | SUCCESS | 0.6248s
25 2025-12-27 18:07:31,139 | INFO | [QUERY] 39ef27b2 | OpenGauss | SUCCESS | 0.0321s
26 2025-12-27 18:07:31,147 | INFO | [QUERY] 2e0eb79f | Neo4j | SUCCESS | 0.0162s
27 2025-12-27 18:07:31,990 | INFO | [QUERY] 41a13958 | Hive | SUCCESS | 0.7843s
28 2025-12-27 18:07:35,038 | INFO | [QUERY] 6d862533 | OpenGauss | SUCCESS | 0.0307s
29 2025-12-27 18:07:35,048 | INFO | [QUERY] 3b02d1b5 | Neo4j | SUCCESS | 0.0225s
30 2025-12-27 18:07:35,750 | INFO | [QUERY] 3024e07e | Hive | SUCCESS | 0.5674s
31 2025-12-27 18:07:38,265 | INFO | [QUERY] e2d9b00b | OpenGauss | SUCCESS | 0.0417s
32 2025-12-27 18:07:38,310 | INFO | [QUERY] 266726b6 | Neo4j | SUCCESS | 0.0491s
33 2025-12-27 18:07:38,955 | INFO | [QUERY] c0a5a113 | Hive | SUCCESS | 0.6116s
34 2025-12-27 18:57:55,102 | INFO | [QUERY] 9d11be4a | Neo4j | SUCCESS | 0.0151s
35 2025-12-27 18:57:55,162 | INFO | [QUERY] c3e2b879 | OpenGauss | SUCCESS | 0.0471s
36 2025-12-27 18:57:55,988 | INFO | [QUERY] 73c53508 | Hive | SUCCESS | 0.7405s
37 2025-12-27 18:58:13,140 | INFO | [QUERY] c3e2b879 | OpenGauss | SUCCESS | 0.0433s
38 2025-12-27 18:58:13,180 | INFO | [QUERY] 9d11be4a | Neo4j | SUCCESS | 0.0279s
39 2025-12-27 18:58:13,220 | INFO | [QUERY] 73c53508 | Hive | SUCCESS | 0.7405s
```

6.小组分工与贡献度

Member	Number	Contribution
王雷	2351299	33.3%
周达	2354185	33.3%
黄景胤	2351129	33.3%